```
"""
Cooper River Trading Co. —
Appraze Billing
--------------------------------
------------
Deliberately uses Stripe PAYMENT
LINKS, not the Checkout Sessions
API or any
card element — consistent with
CRTC's "Stripe Payment Links
only, no raw
card handling" rule. This module
does NOT create checkout
sessions; it only
verifies one after the fact.

Flow:
  1. The Payment Link itself is
created once, by hand, in the
Stripe Dashboard
     (test mode for beta, live
mode when selling for real). Its
URL goes in
```

```
     Streamlit secrets as
STRIPE_PAYMENT_LINK_URL.
  2. In that Payment Link's
settings, "After payment" is set
to redirect to
     this app's URL with ?
session_id={CHECKOUT_SESSION_ID}
appended — Stripe
     fills in the real session id
automatically.
  3. When the browser lands back
on the app with that session_id
in the URL,
     this module makes a read-
only GET to Stripe to confirm
payment_status,
     using the secret key. No
card data ever touches this app.
"""

from dataclasses import dataclass

import requests
import streamlit as st
```

```
@dataclass
class BillingResult:
    paid: bool
    customer_email: str = ""
    error: str = ""


def _secret_key() -> str:
    key =
st.secrets.get("STRIPE_SECRET_KEY
")
    if not key:
        raise
RuntimeError("STRIPE_SECRET_KEY
is not set in Streamlit
secrets.")
    return key


def payment_link_url() -> str:
    return
st.secrets.get("STRIPE_PAYMENT_LI
NK_URL", "")


def
```

```
verify_checkout_session(session_id: str) -> BillingResult:
    """
    Read-only lookup — confirms whether a given Checkout Session (created by
    a Payment Link) actually completed payment. Safe to call repeatedly.
    """
    try:
        resp = requests.get(

f"https://api.stripe.com/v1/checkout/sessions/{session_id}",
            headers=
{"Authorization": f"Bearer {_secret_key()}"},
            timeout=15,
        )
        resp.raise_for_status()
        data = resp.json()
        paid =
data.get("payment_status") ==
"paid"
        email =
```

```
(data.get("customer_details") or
{}).get("email", "")
        return
BillingResult(paid=paid,
customer_email=email)
    except
requests.exceptions.HTTPError as
e:
        return
BillingResult(paid=False,
error=f"Stripe error:
{e.response.status_code}")
    except Exception as e:
        return
BillingResult(paid=False,
error=f"connection error: {e}")
```